

catch

首先可以想到，我们的策略一定是把一些基环树，连向某个原来的基环树 T ，然后在 T 上面抓捕。

这个时候也有两类可能性：如果对 T 也进行了操作，那么一定会把它变成自环。这种情况下，如果操作 k 步，一定能够抓到大小前 k 大的连通块的人。在剩下的情况中，一定不会进行其他操作，否则一定不如上一类更优。

首先枚举连向哪个连通块，记环长为 L ，会首先求出这个块内，找到按距离 $\bmod L$ 分组最多的组。那么对于其他连通块，可以枚举哪个点为根，一定也是把按 $\bmod L$ 分组最多的组连向对应的连通块的位置。直接做可以做到 $O(n^2) \sim O(n^3)$ 的时间复杂度。

接下来考虑优化，首先对于一个连通块（假设环长为 L_2 ），当我们根开始移动的时候，每个点的组大概会从 $dep + pos$ 变成 $dep + pos + L_2$ ，也就是一共会发生 $O(size)$ 次变化，那么可以快速求出每个组的大小以及对应的最大值。如果 $L_2 = L$ ，那么这个值一定是固定的。

进一步发现，这个只和 L 有关，对于每个 L ，会在环长为 L 的连通块里面至少选一个（即最大的），然后对于剩下的块，我们会按权值从大到小选（这部分可以基数排序）。而不同的 L 只会有 $O(\sqrt{n})$ 个，所以总时间复杂度为 $O(n\sqrt{n})$ 。

addone

通过一些观察和样例对拍，我们会发现按任意顺序操作答案都是一样的，我们只需要去模拟这个过程即可。

进一步的思考：

- 中间过程中只关心每个数的奇偶性即可。多出来的部分会放到最后一个数里。
- 每个数的具体数值只会影响最后一个数是否为 0。
- 考虑奇偶性，问题等价于你可以做 `101 -> 000`，`100 -> 001`。

并且最终状态一定为 `...000011111(01)` 即若干个 1 在末尾。因为如果中间有一个 `10`，那么一定可以把前面的 1 消掉。用适当的方法模拟就可以做到 $O(nq)$ 的时间复杂度。

首先解决一下细节问题，即最后一个数字是否为 0，这个可以通过求出最后的奇偶性分布，也最开始的数字总和做比较，可以避免分类讨论，这里维护数字总和可能需要一个树状数组状物。

接着就是考虑这个 01 序列会变成那一类，也就是去分析操作中的不变量。注意根据分析，最后可能有 $2n$ 种等价类。

考虑前缀和，把 10 变成 00，并且把下一位异或 1，相当于把前缀和中的两个相邻的相同的位异或了 1，并且要求和前一位不能相同（但事实上这个关系不大，也就是根据后续分析，如果允许 00 变成 10 并且把下一位异或 1 等价关系还是相同的）。并且要求最后一位有数字，也就是末两位不能修改。考虑前缀和之后，最后的状态是 01 交替，并且末尾不变，也是 $2n$ 种等价类。如果考虑对前缀和序列操作，容易发现奇数位偶数位的差是不变的。所以只需求和就可以还原出前缀和序列的最终状态，即最后的答案。

所以只需要维护前缀和，以及奇偶位置对应的和即可，单点修改对应前缀和区间修改，使用线段树解决即可，时间复杂度为 $O((n + q) \log n)$ 。

但好像也有直接做的做法，根据上述的内容，每个区间一定能消成一个简单的状态，并且和操作顺序无关。我们直接用线段树维护一下每个区间会被消成什么样，然后用讨论一下两个区间如何合并即可。

weight

首先考虑如何做单组询问。考虑贪心，每次优先选择子树中 $a_j < p_j$ 的点中 c_j 最小值，加到整棵子树不小于 q_i 为止。可以发现， a_j 的取值和 c_j 的相对大小有关，和具体的数值关系不大。

接下来把权值分层，即

$$\sum_{i=1}^N c_i a_i = \sum_{k=1}^{C_{\max}} \sum_{i=1}^N [c_i \geq k] \cdot a_i, \quad C_{\max} = \max_i R_i \leq 250.$$

这个时候，如果枚举 k ，只需要考虑尽量少的元素大于等于 k 即可。称 $c_i \geq k$ 为昂贵的， $c_i < k$ 为便宜的。

然后考虑上述的贪心，只需要记录还剩多少便宜的容量即可。记录 $dp_{u,r}$ 表示子树 u 内还剩 r 便宜容量的方案数， $cnt_{u,r}$ 表示子树内还剩 r 便宜容量情况下，使用的昂贵的容量之和。

r 不会超过子树内 $\sum p$ 之和，所以整体 dp 的时间复杂度为 $O((\sum p)^2)$ 。子树合并就是树上背包， u 本身考虑选为便宜和昂贵，按方案数分别转移即可。

时间复杂度 $O(C_{\max}(\sum p)^2)$ 。